

Trees

Dr. Rabins Porwal

Dr. Rabins Porwal

1

Trees

- It is defined as a finite set 'T' of one or more nodes such that there is a node designated as the **root** of the tree and other nodes (excluding the root) are divided into n (≥ 0) disjoint sets $T_1, T_2, \dots, T_n$ and each of these sets is a tree in turn. The trees $T_1, T_2, \dots, T_n$ are called the **sub-trees** or children of the root.

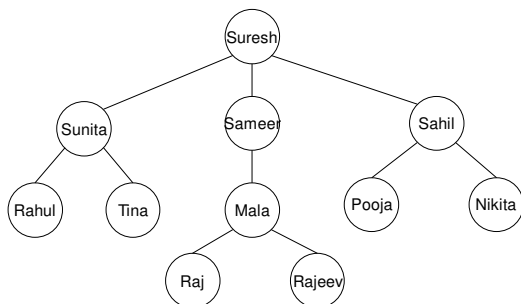
Unit-IV: Trees

2

Dr. Rabins Porwal

2

Example



A Hypothetical Family Tree

Unit-IV: Trees

3

Dr. Rabins Porwal

3

Types of Trees

- Binary Tree
- Binary Search Tree
- Extended Binary Tree (2-Tree)
- Threaded Binary Tree
- AVL Tree (Height Balanced Tree)
- 2-3 Tree
- B-Tree
- Heap
- Forest
- Red Black Tree (RB Tree)

Unit-IV: Trees

4

Dr. Rabins Porwal

4

Binary Tree

- It is a special type of tree in which every node or vertex has **at most** two children, i.e., either no children, one child or two children.
- Alternatively it may also be defined as follows:
 - A binary tree is an empty tree.
 - A binary tree consists of a node called **root**, a **left subtree** and a **right subtree** both of which are binary trees once again.

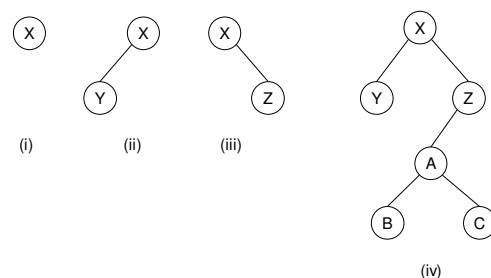
Unit-IV: Trees

5

Dr. Rabins Porwal

5

Examples of Binary Trees



Unit-IV: Trees

6

Dr. Rabins Porwal

6

Applications of Binary Trees

- It is used to represent a non-linear data structure.
- It is used in searching algorithms. Most efficient and commonly used search, known as *Binary search*, uses a special form of binary tree, known as *binary search tree*. *Binary search tree brings down the time complexity of algorithm to less than 50%.*
- Binary trees are used in populating a voluminous, *relational and hierarchical data* into memory which increases the efficiency of the algorithm while managing this data. This is because binary trees allow the algorithms to access a particular node at low cost and binary trees also help to insert a new node easily.
- Binary trees are used in decision making, artificial Intelligence, compilers, expression evaluation, etc.

Unit-IV: Trees

7

Dr. Rabins Porwal

7

Binary Search Tree

- Binary Search Tree**
 - It is a binary tree in which the data elements are stored in such a way that the *key values* in the left subtree of the root are less than the *key value* of the root, and the *key values* of all the nodes in the right subtree of the root are greater than the *key value* of the root. This relationship holds in all the nodes of the tree.
 - This type of structure enables one to search for and find an element with an average running time $O(\log_2 n)$.

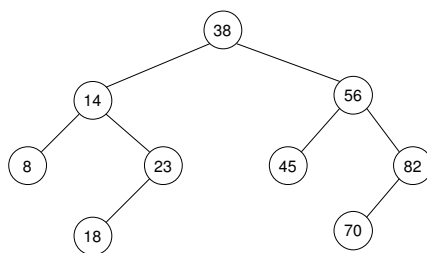
Unit-IV: Trees

8

Dr. Rabins Porwal

8

Example of Binary Search Tree



Unit-IV: Trees

9

Dr. Rabins Porwal

9

Extended Binary Tree

- Extended Binary Tree (2-Trees)**
 - A binary tree can be converted to an extended binary tree by adding new nodes to its leaf nodes, and to the nodes that have only one child. These new nodes are added in such a way that all the nodes in the resultant tree have *either zero or two children*.
 - The nodes in the original tree are called *internal nodes* and the new nodes that are added to binary tree, to make it extended binary tree, are called *external nodes*.

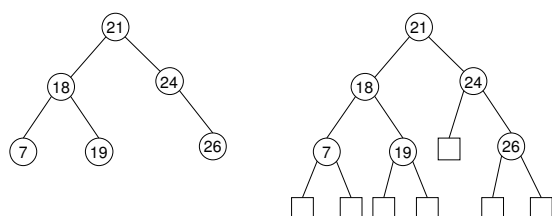
Unit-IV: Trees

10

Dr. Rabins Porwal

10

Example of Extended Binary Tree



Binary Tree converted to an Extended Binary Tree

Unit-IV: Trees

11

Dr. Rabins Porwal

11

Extended Binary Tree (Contd...)

- Points to remember about extended binary tree:**
 - If a tree has n nodes then the number of branches it has is $n - 1$.
 - Every node in a tree has exactly one parent except the root node.
 - A single path connects any two nodes of a tree.
 - For a binary tree of height h the *maximum number of nodes can be $2^{h+1} - 1$* .
 - Any binary tree with n *internal nodes* has $(n+1)$ *external nodes*.

Unit-IV: Trees

12

Dr. Rabins Porwal

12

Some Important Terms

- **Father/ Parent**
- **Leaf Nodes/ Terminal Nodes**
 - A node that has no children is called a leaf node.
- **Ancestor & Descendant**
 - A node n_1 is an ancestor of node n_2 (and n_2 is descendant of n_1) if n_1 is either the father of n_2 or the father of some ancestor of n_2 .
- **Left descendant & Right descendant**
 - A node n_2 is a left descendant of node n_1 if n_2 is either a left son of n_1 or a descendant of the left son of n_1 . A right descendant can be defined similarly as left descendant.
- **Siblings**
 - All the children of a given vertex or node are known as siblings.
- **Strictly Binary Tree**
 - If every non-leaf node in a binary tree has non-empty left and right subtrees.
- **Degree**
 - It is the number of nodes connected to a particular node.

Unit-IV: Trees 13 Dr. Rabins Porwal

13

Some Important Terms (Contd ...)

- **Level of a node**
 - The root of the tree has level 0 and the level of any other node in the tree is one more than the level of its father.
- **Depth/ Height of a Tree**
 - max of (level of all the leaf-nodes of a binary tree)
- **Complete/ Full Binary Tree**
 - All non-leaf nodes have exactly two non-empty children and all leaf nodes are at the same level
- **Internal Path Length**
 - It is defined as the sum of the levels of all the internal nodes in the binary tree. Equivalently, it can also be defined as the sum of length of paths from each node to root.
- **External Path Length**
 - It is defined as the sum of the levels of all the external nodes in the binary tree.

Unit-IV: Trees 14 Dr. Rabins Porwal

14

Properties

- A tree with n nodes has exactly $(n - 1)$ edges or branches.
- In a tree every node except the root has exactly one parent.
- There is exactly one path connecting any two nodes in a tree.
- The maximum number of nodes in a binary tree of height k is $2^{k+1} - 1$, where $k \geq 0$.
- The number of leaf nodes in a complete binary tree of depth d is 2^d .
- A binary tree with n internal nodes has $n + 1$ external nodes.
- If 'E' and 'I' denote the external and internal path lengths of a binary tree having ' n ' internal nodes, then the following identity holds:

$$E = I + 2 \times n$$
- The height of null tree (empty tree) is defined to be -1 .
 - (In this case there is no node in the tree and the start pointer points to NULL)

Unit-IV: Trees 15 Dr. Rabins Porwal

15

Array Representation of Binary Trees

- Let T be a binary tree that is **complete** or **nearly complete**. T can be maintained in the memory using its sequential representation, which uses only a single linear array $TREE$ as follows:
 - The root R of tree T is stored in $TREE[1]$.
 - If a node N occupies $TREE[K]$, then the left child is stored in $TREE[2 \times K]$ and its right child is stored in $TREE[2 \times K + 1]$.
 - $TREE[1] = \text{NULL}$ indicates that the tree is empty.
- Generally, the sequential representation of a tree with depth d requires in array approximately 2^{d+1} elements.
- Therefore, the sequential representation is usually inefficient unless the binary tree is complete or nearly complete.

Unit-IV: Trees 16 Dr. Rabins Porwal

16

Linked Representation of Binary Tree

- Since the definition of this data structure is recursive, an appropriate representation could be self-referential structure.
- Each node of a tree is represented by a structure.
- Each node contains two links to the same structure – left & right.
- If tree is NULL, the tree is empty otherwise tree points to the root node of the tree, left link points to the left subtree of the tree and right link points to the right subtree of the tree.
- The structure used to represent binary tree in a linked list is:


```
struct node{
    struct node * left;
    int data;
    struct node * right;
};
```

Unit-IV: Trees 17 Dr. Rabins Porwal

17

Binary Tree Traversal

- **Recursive Traversal of Binary Tree:**
 - Traversing is the process of visiting every node in the tree exactly once. Therefore, a complete traversal of binary tree implies visiting the nodes of the tree in some linear sequence
 - Three standard ways of traversing a binary tree
 - Preorder or depth-first order (DLR / NLR)
 - Inorder or symmetric order (LDR / LNR)
 - Postorder (LRD / LRN)

Depth-First Traversals

Unit-IV: Trees 18 Dr. Rabins Porwal

18



Inorder Traversal



- Three steps:
 - Traverse the left subtree **in inorder**
 - Visit the root
 - Traverse the right subtree **in inorder**
- Algorithm

```
Inorder (node)
1. If node is not equal to NULL [Do through step 4]
2. Call inorder (left-child[node])
3. Output data[node]
4. Call inorder (right-child[node])
5. Exit
```

Unit-IV: Trees

19

Dr. Rabins Porwal

19



Preorder Traversal



- Three steps:
 - Visit the root
 - Traverse the left subtree **in preorder**
 - Traverse the right subtree **in preorder**
- Algorithm

```
Preorder (node)
1. If node is not equal to NULL [Do through step 4]
2. Output data[node]
3. Call preorder (left-child[node])
4. Call preorder (right-child[node])
5. Exit
```

Unit-IV: Trees

20

Dr. Rabins Porwal

20



Postorder Traversal



- Three steps:
 - Traverse the left subtree **in postorder**
 - Traverse the right subtree **in postorder**
 - Visit the root
- Algorithm

```
Postorder (node)
1. If node is not equal to NULL [Do through step 4]
2. Call postorder (left-child[node])
3. Call postorder (right-child[node])
4. Output data[node]
5. Exit
```

Unit-IV: Trees

21

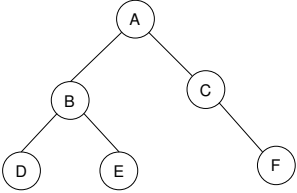
Dr. Rabins Porwal

21



Example





- Inorder Traversal: DBEACF
- Preorder Traversal: ABDECF
- Postorder Traversal: DEBFCA

Unit-IV: Trees

22

Dr. Rabins Porwal

22

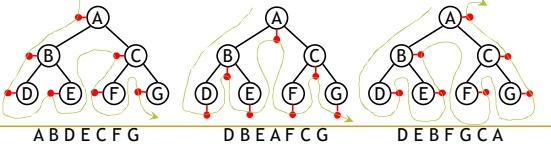


Tree Traversals using “flags”



- The order in which the nodes are visited during a tree traversal can be easily determined by imagining there is a “flag” attached to each node, as follows:


- To traverse the tree, collect the flags:



Unit-IV: Trees

23

Dr. Rabins Porwal

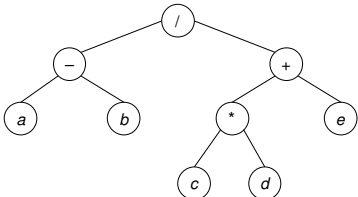
23



Algebraic Expressions



- Every algebraic expression corresponds to a unique tree and vice versa.
- Example:** Consider the following expression
$$E = (a - b) / ((c * d) + e)$$



Unit-IV: Trees

24

Dr. Rabins Porwal

24

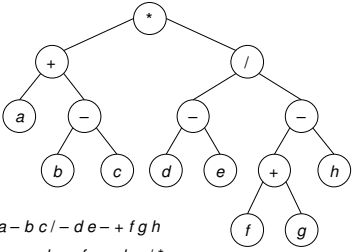


Applications of Binary Tree Traversal



■ Let $E = [a + (b - c)] * [(d - e) / (f + g - h)]$

This can be represented as following binary tree



```
graph TD
    Root((*)) --- L1_1((+))
    Root --- L1_2((/))
    L1_1 --- L2_1((a))
    L1_1 --- L2_2((-))
    L2_2 --- L3_1((b))
    L2_2 --- L3_2((c))
    L1_2 --- L2_3((-))
    L1_2 --- L2_4((-))
    L2_3 --- L3_3((d))
    L2_3 --- L3_4((e))
    L2_4 --- L3_5((+))
    L3_5 --- L3_6((f))
    L3_5 --- L3_7((g))
    L3_5 --- L3_8((h))
```

Preorder: * + a - b c / - d e - + f g h

Postorder: a b c - + d e - f g + h - / *

Unit-IV: Trees

25

Dr. Rabins Porwal

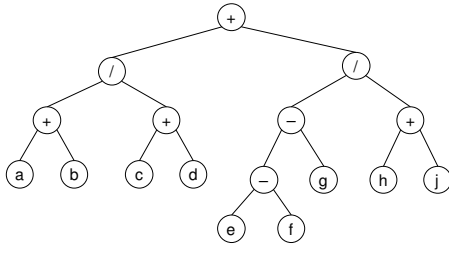
25



More Examples



■ Let $E = (a + b) / (c + d) + (e - f - g) / (h + j)$



```
graph TD
    Root((+)) --- L1_1((/))
    Root --- L1_2((/))
    L1_1 --- L2_1((+))
    L1_1 --- L2_2((+))
    L2_1 --- L3_1((a))
    L2_1 --- L3_2((b))
    L2_2 --- L3_3((c))
    L2_2 --- L3_4((d))
    L1_2 --- L2_3((-))
    L1_2 --- L2_4((+))
    L2_3 --- L3_5((-))
    L3_5 --- L3_6((e))
    L3_5 --- L3_7((f))
    L3_5 --- L3_8((g))
    L2_4 --- L3_9((h))
    L2_4 --- L3_10((j))
```

Preorder: + / + a b + c d / - - e f g + h j

Postorder: a b + c d + / e f - g - h j + / +

Unit-IV: Trees

26

Dr. Rabins Porwal

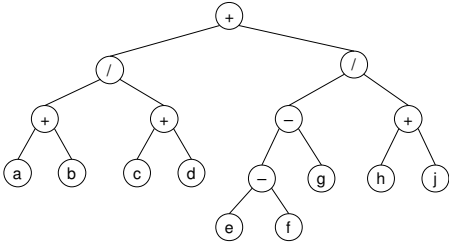
26



Construction of Tree from the Prefix expression



■ Prefix Expression: + / + a b + c d / - - e f g + h j



```
graph TD
    Root((+)) --- L1_1((/))
    Root --- L1_2((/))
    L1_1 --- L2_1((+))
    L1_1 --- L2_2((+))
    L2_1 --- L3_1((a))
    L2_1 --- L3_2((b))
    L2_2 --- L3_3((c))
    L2_2 --- L3_4((d))
    L1_2 --- L2_3((-))
    L1_2 --- L2_4((+))
    L2_3 --- L3_5((-))
    L3_5 --- L3_6((e))
    L3_5 --- L3_7((f))
    L3_5 --- L3_8((g))
    L2_4 --- L3_9((h))
    L2_4 --- L3_10((j))
```

Unit-IV: Trees

27

Dr. Rabins Porwal

27



Threaded Binary Tree



■ Any drawback with the binary tree?

□ Too many null pointers in current representation of binary trees

■ n : number of nodes

number of non-null links: $n - 1$

total links: $2n$

null links: $2n - (n - 1) = n + 1$

■ A. J. Perils & C. Thornton jointly proposed idea to make effective use of these null pointers.

□ According to this idea, replace all the null pointers with some useful values called "threads".

Unit-IV: Trees

28

Dr. Rabins Porwal

28



Threaded Binary Trees: Classification



■ Based on left/ right thread

□ Left-threaded

□ Right-threaded

□ Fully Threaded

■ Based on traversal order

□ Preorder threaded

□ In-order threaded

□ Post-order threaded

Unit-IV: Trees

29

Dr. Rabins Porwal

29



Threaded Binary Tree



■ One may choose a *one-way threading* or a *two-way threading*. Here, our threading will correspond to the *inorder traversal* of T.

□ Accordingly, in the one way threading of T, a thread will appear in the right field of a node and will point to the next node (successor) in the in-order traversal (*inorder successor*) of T.

Unit-IV: Trees

30

Dr. Rabins Porwal

30

Threaded Binary Tree (One-Way)

Inorder of the tree is: 1, 3, 5, 6, 7, 8, 9, 11, 13

Unit-IV: Trees

31

Dr. Rabins Porwal

31

Threaded Binary Tree (Two-way)

- In this type of threaded tree, the left pointer of a node 'n', which is NULL as its left child is an empty subtree, is said to point to preceding node in the in-order traversal (*inorder predecessor*) of 'n' and the right pointer of a node 'm', which is NULL as its right child is an empty subtree, is said to point to the next node in the in-order traversal (*inorder successor*) of 'm'.
- The links with arrowheads (known as *thread*) indicate links leading to inorder predecessor or inorder successor while *lines* denote the usual links in a binary tree.

Unit-IV: Trees

32

Dr. Rabins Porwal

32

Example (Two-Way) Threaded B. Tree

Consider D & G – the left and right links keep dangling because they do not have any inorder predecessor or successor. To get rid of such problems, introduce a *header node*.

Inorder Traversal: D J H K B E A F C I G

Unit-IV: Trees

33

Dr. Rabins Porwal

33

Implementation Issues

- The structure of a node in such a tree is:

```
struct bthreadnode {
    int leftflag, rightflag;
    int data;
    struct bthreadnode *left, *right;
};
```
- Two flags '*leftflag*' and '*rightflag*' are used per node to indicate the type of its 'left' and 'right' links.
- If '*leftflag*' of a node '*n*' is 0, then its left link leads to the *left subtree* of '*n*', otherwise to the *inorder predecessor* of '*n*'.
- Similarly, if '*rightflag*' of a node '*n*' is 0, then its right link leads to the *right subtree* of '*n*', otherwise to the *inorder successor* of '*n*'.

Unit-IV: Trees

34

Dr. Rabins Porwal

34

Threaded Tree Traversal (In-order)

- We start at the leftmost node in the tree, print it, and follow its right thread
 - If we follow a *thread* to the right, we *output the node and continue to its right*.
 - If we follow a *link* to the right, we *go to the leftmost node, print it, and continue*.

Unit-IV: Trees

35

Dr. Rabins Porwal

35

Threaded Tree Traversal

Output
1

Start at leftmost node, print it

Unit-IV: Trees

36

Dr. Rabins Porwal

36

Threaded Tree Traversal

Output
1
3

Follow thread to right, print node

Unit-IV: Trees37Dr. Rabins Porwal

37

Threaded Tree Traversal

Output
1
3
5

Follow link to right, go to leftmost node and print

Unit-IV: Trees38Dr. Rabins Porwal

38

Threaded Tree Traversal

Output
1
3
5
6

Follow thread to right, print node

Unit-IV: Trees39Dr. Rabins Porwal

39

Threaded Tree Traversal

Output
1
3
5
6
7

Follow link to right, go to leftmost node and print

Unit-IV: Trees40Dr. Rabins Porwal

40

Threaded Tree Traversal

Output
1
3
5
6
7
8

Follow thread to right, print node

Unit-IV: Trees41Dr. Rabins Porwal

41

Threaded Tree Traversal

Output
1
3
5
6
7
8
9

Follow link to right, go to leftmost node and print

Unit-IV: Trees42Dr. Rabins Porwal

42

Threaded Tree Traversal

Output

```

1
3
5
6
7
8
9
11

```

Follow thread to right, print node

Unit-IV: Trees 43 Dr. Rabins Porwal

43

Threaded Tree Traversal

Output

```

1
3
5
6
7
8
9
11
13

```

Follow link to right, go to leftmost node and print

Unit-IV: Trees 44 Dr. Rabins Porwal

44

Threaded Tree Traversals

- Similarly, can also be employed to *pre-order* and *post-order* traversals
 - As Threaded Tree can be threaded using any order traversal
- Useful in applications which require
 - Information about predecessor & successor nodes
 - efficient order traversals

Unit-IV: Trees 45 Dr. Rabins Porwal

45

Threaded Tree Modification

- We're still wasting pointers, since half of our leafs' pointers are still null
- We can add threads to the previous node in an inorder traversal as well, which can be used to traverse the tree backwards or even to do postorder traversals

Unit-IV: Trees 46 Dr. Rabins Porwal

46

Threaded Tree Modification

Unit-IV: Trees 47 Dr. Rabins Porwal

47

Advantages

- Threaded binary trees have numerous advantages over non-threaded binary trees:
 - The traversal operation is much faster than that of its unthreaded version, because with threaded binary tree *non-recursive implementation is possible which can run faster* and does not require the botheration of stack management.
 - General operations like traversal, insertion, deletion can be done without using recursive algorithms & stacks.

Unit-IV: Trees 48 Dr. Rabins Porwal

48

Advantages (Contd...)

- In threaded binary tree, we can *efficiently determine the predecessor and successor nodes* starting from any node. In case of unthreaded binary tree, however, this task is more time consuming and difficult. For this case, a stack is required to provide upward pointing information in the tree whereas in a threaded binary tree, without having to include the overhead of using a stack mechanism the same can be carried out with the threads.

Unit-IV: Trees 49 Dr. Rabins Porwal

49

Advantages (Contd...)

- In threaded binary tree, any node can be accessed from any other node. *Threads are usually move to upward whereas links are downward*. Thus, in a threaded tree, one can move in either direction and nodes are in fact circularly linked. This is not possible in unthreaded counter part because there we can move only in downward direction starting from root.
- Insertion into and deletions from a threaded tree are although time consuming operations but these are very easy to implement.

Unit-IV: Trees 50 Dr. Rabins Porwal

50

Advantages (Contd...)

- A threaded binary tree, due to its threaded nature, reduces the time complexity of algorithms.

Unit-IV: Trees 51 Dr. Rabins Porwal

51

Disadvantages

- Insertion into and deletion from a threaded tree are very *time-consuming* operations in comparison to non-threaded binary tree.
- This tree require additional bit to identify the threaded link.

Unit-IV: Trees 52 Dr. Rabins Porwal

52

Number of Binary Trees

- The total number of distinct binary trees using n nodes can be found by using the following formula:

$$2^n C_n - 2^n C_{n-1}$$

Unit-IV: Trees 53 Dr. Rabins Porwal

53

Reconstruction of Binary Tree

- The original tree cannot be reconstructed, given its inorder, preorder or postorder traversal alone.
- If sequence of nodes produced by inorder and postorder traversal of a binary tree are provided, then a unique binary tree can be reconstructed.

Unit-IV: Trees 54 Dr. Rabins Porwal

54

Example

- Inorder: HDIJEKKBALFMCNGO
- Postorder: HIDJKEBLMFNOGCA

Unit-IV: Trees 55 Dr. Rabins Porwal

55

Path Length

- In every external binary tree (extended binary tree), number of external nodes is 1 more than number of internal nodes.
$$E = I + 1$$
- The **path length** for any node is the minimum number of nodes traversed from root to that node.
- Total path length = sum of all path lengths
- The total path length of external nodes can be obtained by
$$P_E = P_I + 2 \cdot n,$$
where n is the total number of internal nodes in the tree.

Unit-IV: Trees 56 Dr. Rabins Porwal

56

Weighted Path Length

- Assume, every external node has some weight ' w ', then **weighted path length** for external nodes will be:
$$P = w_1 P_1 + w_2 P_2 + \dots + w_n P_n$$
where w_i denotes the weight and P_i denotes the path length of an external node n_i .
- Problem:** Different trees which have same weights on external nodes then **it is not necessary** that they have same weighted path length.

Unit-IV: Trees 57 Dr. Rabins Porwal

57

Example

- Consider three possibilities for weights 4, 7, 8, 12 on external nodes.

$P_a = 4 \times 2 + \dots = 62$
 $P_b = 4 \times 1 + \dots = 77$
 $P_c = 12 \times 1 + \dots = 61$

Unit-IV: Trees 58 Dr. Rabins Porwal

58

Problem & Solution

- Problem:** Obtain a unique tree which has **minimum weighted path length**.
- Solution:** **Huffman Algorithm**, which can be used in obtaining extended binary tree.
- Huffman Algorithm
 - Let us consider, there are n weights $w_1, w_2, \dots, w_n$.
 - Take two minimum weights and create a subtree. Suppose w_1 & w_2 are first two minimum weights.
 - Remaining weights will be $w_1 + w_2, w_3, w_4, \dots, w_n$.
 - Repeat the process of creating all subtrees upto last weight and thus we get a tree having minimum weighted path length.

Unit-IV: Trees 59 Dr. Rabins Porwal

59

Example

- Draw an extended binary tree for the following weights, which has minimum weighted path length:
(1) 16 11 7 20 25 5 16
(2) 13 54 12 66 87 35 16 60 23 75

Unit-IV: Trees 60 Dr. Rabins Porwal

60